

EXPERIMENT-6

Student Details

Name: R.Varshitha

Roll Number: 24501a4450

Dataset: student_performance.csv

1. Business Understanding

1.1 Problem Statement & Relevance

Student performance can be classified into different Grade categories using student demographic and academic features.

In this experiment, a Decision Tree Classifier is developed to predict Grade using Age, Gender, Study_Hours, Attendance(%) and Test_Score.

Problem Statement

Develop a machine learning model that predicts a student's Grade based on student performance features.

Relevance

- Helps classify student performance into Grade categories.
- Supports teachers and institutions in understanding academic performance.
- Uses decision rules based on student performance features.
- Can help identify student performance categories.

1.2 Expected Benefits

- Better classification of student performance levels.
- Uses a Decision Tree to learn feature-based classification rules.
- Supports academic performance analysis.
- Provides a visual Decision Tree for the learned classification rules.

2. Data Understanding

2.1 Dataset Structure

Dataset Name: student_performance.csv

Variables used by the supplied program:

- Age, Gender, Study_Hours, Attendance(%), Test_Score
- Grade

The supplied program checks the dataset structure, descriptive statistics, shape, duplicate records and missing values.

2.2 Data Dictionary

Column	Description	Type	Role
Age	Student's age	Numerical	Independent feature
Gender	Student's gender	Categorical	Independent feature

Study_Hours	Hours spent studying	Numerical	Independent feature
Attendance(%)	Attendance percentage	Numerical	Independent feature
Test_Score	Student test score	Numerical	Independent feature
Grade	Student performance grade	Categorical	Target variable

2.3 Data Quality Observations

The program performs basic data inspection and cleaning before modelling.

Source Code:

```
df.head()

df.info()

df.describe()

df.shape

print("No of duplicate records:", df.duplicated().sum())

# Remove duplicate records
df = df.drop_duplicates()

print("No of duplicate records after removal:", df.duplicated().sum())

df.isnull().sum()
```

Observations

- Missing values are checked using `df.isnull().sum()`.
- Duplicate records are checked and removed using `df.drop_duplicates()`.
- Numerical and categorical features are identified using `select_dtypes()`.
- The Decision Tree is displayed using the supplied `tree.plot_tree()` code.

3. Data Preparation

3.1 Data Cleaning

The following preprocessing steps are present in the supplied program:

- Load the Student Performance dataset.
- Inspect dataset information and descriptive statistics.
- Check missing values.
- Check duplicate records.
- Remove duplicate records.
- Identify numerical and categorical features.

Source Code:

```
df = pd.read_csv('/content/student_performance.csv')

df.head()

df.info()

df.describe()

df.shape

print("No of duplicate records:", df.duplicated().sum())

# Remove duplicate records
df = df.drop_duplicates()

print("No of duplicate records after removal:", df.duplicated().sum())

df.isnull().sum()
```

3.2 Feature Engineering

Independent Features (X)

Age, Gender, Study_Hours, Attendance(%), Test_Score

Source Code:

```
x = df.drop('Grade', axis=1)
```

Dependent Variable (Y)

Grade

Source Code:

```
y = df['Grade']
```

Encoding

Gender is a categorical feature. The supplied program applies LabelEncoder to Gender separately on the training and testing feature data.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

x_train_encoded = x_train.copy()
x_test_encoded = x_test.copy()

x_train_encoded['Gender'] = le.fit_transform(x_train_encoded['Gender'])
x_test_encoded['Gender'] = le.transform(x_test_encoded['Gender'])
```

3.3 Train-Test Split

Training Data = 80%

Testing Data = 20%

Random State = 42, stratified by Grade

Source Code:

```
x_train, x_test, y_train, y_test = train_test_split(
    x_cleaned, y_cleaned,
    test_size=0.2,
    random_state=42,
    stratify=y_cleaned
)
```

4. Modelling

4.1 Model Selection

Algorithm Used:

Decision Tree Classifier

Reason:

Decision Tree Classification is suitable because Grade is a categorical target. The model learns decision rules from the student performance features and classifies students into Grade categories.

Criterion Used:

criterion='entropy'

4.2 Model Training

Steps:

- Import required libraries.
- Load and inspect the dataset.
- Remove duplicate rows.
- Separate Grade as Y and the remaining columns as X.
- Remove rows containing missing values before the train-test split.
- Split the data into training and testing sets.
- Encode Gender in the training and testing feature data.
- Train the Decision Tree Classifier and generate predictions.

Source Code:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Instantiate LabelEncoder
le = LabelEncoder()

# Apply LabelEncoder to the 'Gender' column in x_train and x_test
# Fit on x_train and transform both x_train and x_test to ensure consistency
x_train_encoded = x_train.copy()
x_test_encoded = x_test.copy()

x_train_encoded['Gender'] = le.fit_transform(x_train_encoded['Gender'])
x_test_encoded['Gender'] = le.transform(x_test_encoded['Gender'])

model = DecisionTreeClassifier(criterion='entropy')

model.fit(x_train_encoded, y_train)
```

```
# predictions
y_pred = model.predict(x_test_encoded)
print(y_pred)
```

4.3 Model Outputs & Interpretation

The program predicts Grade values for the test data using the trained Decision Tree Classifier.

Source Code:

```
y_pred = model.predict(x_test_encoded)
print(y_pred)
```

The model predictions are evaluated using a confusion matrix, classification report and accuracy score.

5. Evaluation

5.1 Appropriate Metrics Selection

The supplied program calculates classification evaluation metrics: confusion matrix, classification report and accuracy. It also calculates training and testing accuracy using `model.score()`.

- Confusion Matrix - shows the number of correct and incorrect predictions for each class.
- Classification Report - displays precision, recall and F1-score for the Grade classes.
- Accuracy Score - measures the proportion of correctly classified test observations.
- Training and Testing Accuracy - measures model performance on the training and test datasets.

Source Code - Evaluation:

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))
```

Source Code - Training and Testing:

```
train_accuracy = model.score(x_train_encoded, y_train)

test_accuracy = model.score(x_test_encoded, y_test)

print("Training Accuracy: {:.2f}%".format(train_accuracy * 100))
print("Testing Accuracy: {:.2f}%".format(test_accuracy * 100))
```

5.2 Interpretation

- Higher accuracy indicates that more observations are classified correctly.
- The confusion matrix shows the distribution of correct and incorrect predictions.
- Precision, recall and F1-score provide class-wise evaluation in the classification report.
- Training and testing accuracy are compared to understand how well the model generalizes.

Training Accuracy: 100.00%

Testing Accuracy: 100.00%

6. Conclusion

The Student Performance dataset is modelled using a Decision Tree Classifier with entropy criterion. The model uses Age, Gender, Study_Hours, Attendance(%) and Test_Score to predict Grade. The supplied program checks and removes duplicate records, checks missing values, splits the data into training and testing sets, encodes Gender, trains the model, generates predictions and evaluates the model using a confusion matrix, classification report, accuracy score, training accuracy and testing accuracy.

Future improvements include:

- Compare the Decision Tree with other classification algorithms.
- Use cross-validation for model evaluation.
- Tune Decision Tree parameters to reduce possible overfitting.
- Use additional student performance features and more unseen data for validation.

7. Appendix

Decision Tree Visualization

Source Code:

```
from sklearn import tree
import matplotlib.pyplot as plt

plt.figure(figsize=(15, 15))

tree.plot_tree(
    model,
    feature_names=x.columns,
    filled=True
)

plt.show()
```

The Decision Tree image is provided separately by the student and can be inserted here.

